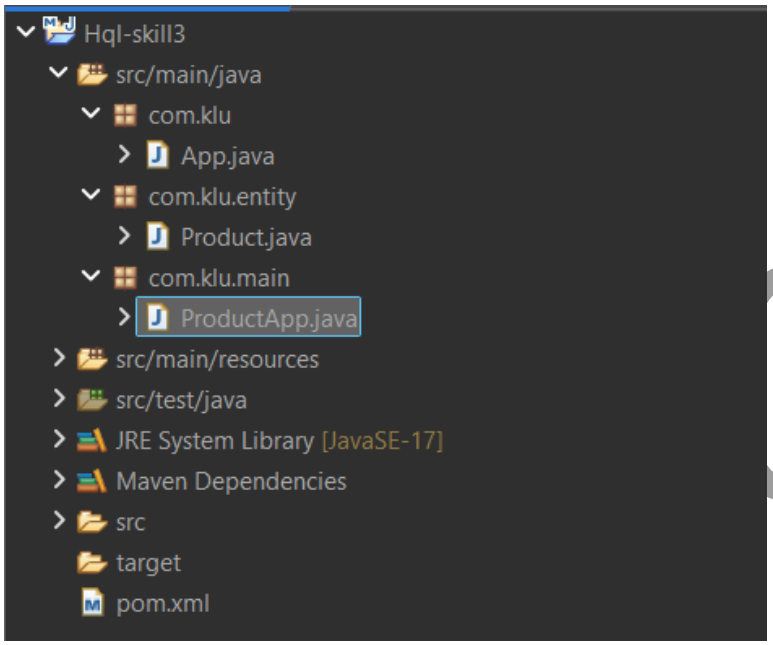
SKILL-10: Working with HQL - Sorting, Pagination & Aggregates

ID: 2400030128
Name: M. Naga Subrahmanyam

Section: 2

In this lab, you will extend the Hibernate module created in Skill 2 and perform advanced data retrieval using HQL, including sorting, pagination, filtering, and aggregate functions. These operations help in retrieving structured and meaningful data from the database using Hibernate.

Structure:



0128

Codes:

Product.java:

```
package com.klu.entity;



import jakarta.persistence.Entity;

import jakarta.persistence.Id;

import jakarta.persistence.Table;


@Entity
```

```
@Table(name="product_skill3")
public class Product
{
   @Id
   private int id;
   private String name;
   private String description;
   private double price;
   private int quantity;

   // Getters and Setters

   public int getId() {
      return id;
   }

   public void setId(int id) {
      this.id = id;
   }

   public String getName() {
      return name;
   }

   public void setName(String name) {
      this.name = name;
   }

   public String getDescription() {
      return description;
   }
```

2400030128

```
    public void setDescription(String description) {
        this.description = description;
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public int getQuantity() {
        return quantity;
    }

    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }
}
```

2400030128

ProductApp.java:

```
package com.klu.main;


import java.util.List;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
import org.hibernate.query.Query;
```

```
import com.klu.entity.Product;

public class ProductApp
{
   public static void main(String[] args)
   {
      Configuration cfg = new Configuration().configure("hibernate.cfg.xml");
      SessionFactory sf = cfg.buildSessionFactory();
      Session session = sf.openSession();

      // 1. Sorting by price ASC
      Query<Product> q1 = session.createQuery("from Product p order by p.price asc",
Product.class);
      List<Product> list1 = q1.list();
      System.out.println("Products sorted by price ASC:");
      list1.forEach(p -> System.out.println(p.getName()+" "+p.getPrice()));

      // 2. Sorting by price DESC
      Query<Product> q2 = session.createQuery("from Product p order by p.price desc",
Product.class);
      List<Product> list2 = q2.list();
      System.out.println("Products sorted by price DESC:");
      list2.forEach(p -> System.out.println(p.getName()+" "+p.getPrice()));

      // 3. Sort by quantity highest first
      Query<Product> q3 = session.createQuery("from Product p order by p.quantity desc",
Product.class);
      List<Product> list3 = q3.list();
      System.out.println("Products sorted by quantity:");
      list3.forEach(p -> System.out.println(p.getName()+" "+p.getQuantity()));
```

2400030128

```
// 4. Pagination - First 3
Query<Product> q4 = session.createQuery("from Product", Product.class);
q4.setFirstResult(0);
q4.setMaxResults(3);
List<Product> page1 = q4.list();

System.out.println("First 3 Products:");
page1.forEach(p -> System.out.println(p.getName()));

// 5. Pagination - Next 3
Query<Product> q5 = session.createQuery("from Product", Product.class);
q5.setFirstResult(3);
q5.setMaxResults(3);
List<Product> page2 = q5.list();

System.out.println("Next 3 Products:");
page2.forEach(p -> System.out.println(p.getName()));

// 6. Aggregate Functions

// Count total products
Query<Long> q6 = session.createQuery("select count(p.id) from Product p", Long.class);
System.out.println("Total Products: "+q6.uniqueResult());

// Count products where quantity > 0
Query<Long> q7 = session.createQuery("select count(p.id) from Product p where p.quantity > 0", Long.class);
System.out.println("Products with quantity >0: "+q7.uniqueResult());

// Minimum price
Query<Double> q8 = session.createQuery("select min(p.price) from Product p", Double.class);
```

2400030128

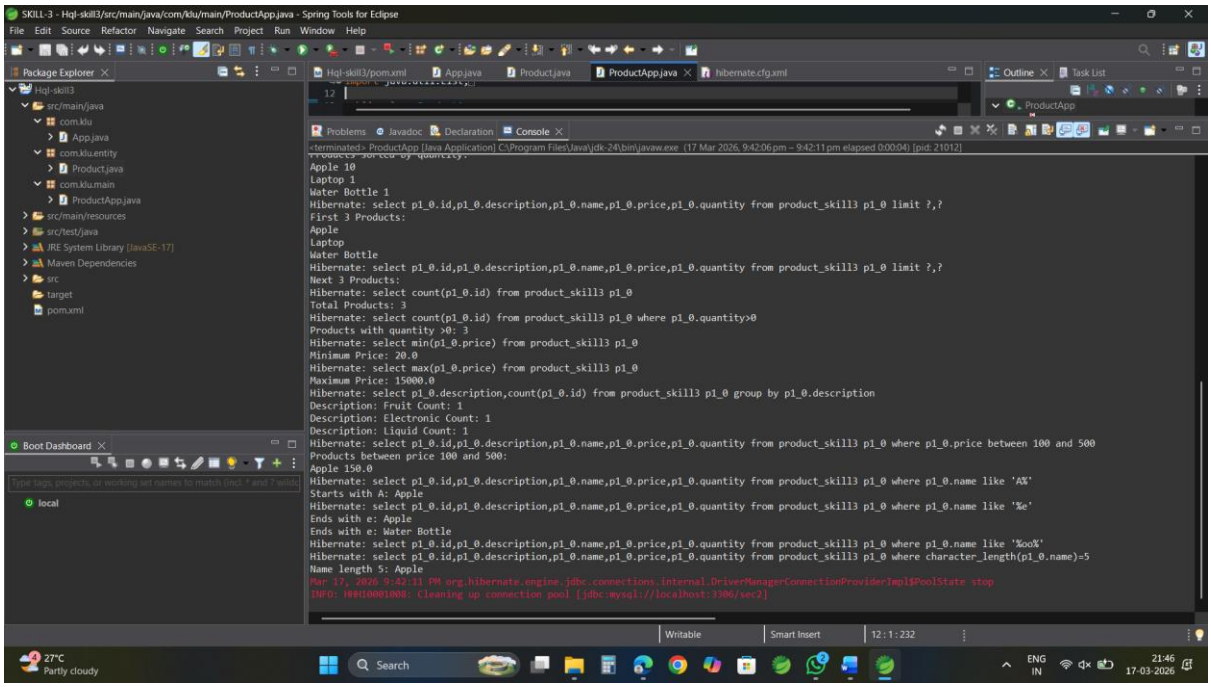
```
System.out.println("Minimum Price: "+q8.uniqueResult());

// Maximum price
Query<Double> q9 = session.createQuery("select max(p.price) from Product p",
Double.class);
System.out.println("Maximum Price: "+q9.uniqueResult());

// 7. GROUP BY description
Query<Object[]> q10 = session.createQuery(
    "select p.description, count(p.id) from Product p group by p.description",
    Object[].class);

List<Object[]> groupList = q10.list();

for(Object[] row : groupList)
{
  System.out.println("Description: "+row[0]+" Count: "+row[1]);
}

// 8. WHERE clause price range
Query<Product> q11 = session.createQuery(
    "from Product p where p.price between 100 and 500", Product.class);

List<Product> priceRange = q11.list();

System.out.println("Products between price 100 and 500:");
priceRange.forEach(p -> System.out.println(p.getName()+" "+p.getPrice()));

// 9. LIKE queries

// Names starting with 'A'
```

2400030128

```
    Query<Product> q12 = session.createQuery(
        "from Product p where p.name like 'A%'", Product.class);

    q12.list().forEach(p -> System.out.println("Starts with A: "+p.getName()));

    // Names ending with 'e'
    Query<Product> q13 = session.createQuery(
        "from Product p where p.name like '%e'", Product.class);

    q13.list().forEach(p -> System.out.println("Ends with e: "+p.getName()));

    // Names containing 'oo'
    Query<Product> q14 = session.createQuery(
        "from Product p where p.name like '%oo%'", Product.class);

    q14.list().forEach(p -> System.out.println("Contains oo: "+p.getName()));

    // Exact length (example 5 characters)
    Query<Product> q15 = session.createQuery(
        "from Product p where length(p.name)=5", Product.class);

    q15.list().forEach(p -> System.out.println("Name length 5: "+p.getName()));

    session.close();
    sf.close();
  }
}
```

240003012 8

OUTPUT:



2400030128

GITHUB LINK:

https://github.com/nagasubrahmanyammeka/SkillExperiments_2400030128_FSAD_Sec2/tree/main/SKILL-3